

Lab 2A: The Magic Moment

Duration: 30-45 min | **Difficulty:** Beginner

After: Installation & Agentic Loop slides

© 2026 AIA Copilot — Instructor-Led Training Material

Goal

Go from an empty folder to a running web app in a single prompt. This is the moment that changes how you think about building software.

Pre-Flight Check (5 min)

Open a terminal and verify your environment:

Which terminal? Use whatever you normally use — PowerShell, Windows Terminal, Mac Terminal, or Linux shell. Claude Code works in all of them.

```
node -v
```

Expected: `v18.x.x` or higher

```
git --version
```

Expected: `git version 2.x.x`

```
claude --version
```

Expected: A version number like `1.x.x`

Not installed? See the Installation slide or ask your instructor.

☐ Pre-Flight Complete

- ☐ Node.js 18+ installed
- ☐ Git installed
- ☐ Claude Code installed

Steps

1. Create Your Project and Start Claude

```
mkdir business-dashboard  
cd business-dashboard  
claude
```

You're now in an interactive Claude Code session. **From this point forward, everything happens inside Claude.**

2. The Build Prompt

Copy and paste this prompt exactly:

Initialize this as a git repo and build me a business dashboard web application:

Tech stack: Node.js, Express, SQLite (via better-sqlite3), vanilla HTML/CSS/JS frontend
No external CSS frameworks – write clean, modern CSS

Database tables:

1. metrics – id, name, value (number), category, recorded_at
2. tasks – id, title, status (pending/in-progress/completed), priority (high/medium/low), assigned_to, created_at, completed_at

API endpoints:

- GET /api/metrics (filter by ?category=)
- POST /api/metrics
- GET /api/tasks (filter by ?status= and ?priority=)
- POST /api/tasks
- PUT /api/tasks/:id
- DELETE /api/tasks/:id
- GET /api/dashboard (summary: all metrics + task counts by status + high priority tasks)

Frontend: Dark theme dashboard at / showing all metrics as cards and tasks as a list.
Seed the database with 8 sample metrics across categories (financial, customers, operations, hr) and 5 sample tasks.

Initialize npm, install dependencies, create everything, and start the server. Use port 3100 (or PORT env var).

Include a .gitignore for node_modules and *.db files.

Watch Claude work through the agentic loop: 1. **Plan** — Analyzes requirements, decides on file structure 2. **Execute** — Creates files, runs `npm install`, writes code 3. **Validate** — Checks for errors, tests the server

***Instructor Note:** This is the "magic moment." Let it run — don't interrupt.*

3. See It in the Browser

Open your browser to <http://localhost:3100>

You should see a dark-themed dashboard with 8 metric cards and 5 tasks.

4. Test the API Through Claude

Test the API: add a new metric called "New Leads This Week" with value 42 in the "sales" category. Then add a task "Review Q2 projections" with high priority assigned to me. Then complete task 1.
Show me the results after each operation.

Refresh the dashboard in your browser. The new data should appear.

5. Your First Git Commit

Commit everything with the message "feat: initial business dashboard with API and UI"

*You may see a **Skills** prompt. If Claude asks to use a "commit" skill, select **Yes** — this is normal. You'll learn more about Skills in Day 3.*

What Just Happened?

This is the **agentic loop** in action:

1. **Planning:** Claude read your prompt, understood the requirements, and created a mental plan
2. **Execution:** It autonomously created files, ran npm commands, wrote code
3. **Validation:** It checked that the structure made sense before presenting it to you

You didn't tell Claude *how* to build it. You told it *what* you wanted. Claude: - Knew to use Express (industry standard for Node.js APIs) - Chose better-sqlite3 (fast, zero-config database) - Structured the code properly (server setup, routes, database layer) - Added error handling without being asked - Seeded test data so the dashboard isn't empty - Built a complete frontend with dark theme styling

This is fundamentally different from autocomplete or code snippets. Claude understood the **intent** and delivered a **working system**.

Vague vs. Specific Prompting

Try this vague prompt first:

Add validation.

Look at what Claude does — it guesses which routes, which fields, what error format.

Now try the specific version:

```
In @src/index.js, modify only the POST /api/tasks route. Validate:
- title: must be non-empty string, max 200 characters
- priority: must be one of "high", "medium", "low"
- assigned_to: optional, but if provided must be non-empty string
Return 400 with { error: "description of what's wrong" } for invalid requests.
```

Compare the outputs. **The specific prompt produces exactly what you want on the first try.**

***Specificity isn't extra work — it's less work.** One precise prompt beats three vague ones.*

Optional: Go Deeper

Add more endpoints:

```
Add these endpoints to the API:
- GET /api/metrics/summary – returns min, max, average value per category
- GET /api/tasks/overdue – returns tasks that are pending and older than 7 days
- PATCH /api/tasks/:id/assign – changes the assigned_to field only
Test each one and show me the results.
```

Explore the code Claude wrote:

```
Show me the file structure of this project and explain each file's purpose
```

Try breaking something on purpose:

```
What happens if I POST a task with no title? Show me the error response.
```

□ Lab 2A Checkpoint

- ☐ A running Express server on port 3100
- ☐ SQLite database with metrics and tasks
- ☐ Working web UI at <http://localhost:3100>
- ☐ Input validation on POST /api/tasks

- [] 7+ API endpoints all working
- [] First git commit

Keep Claude running. Return to slides when your instructor is ready.

© 2026 AIA Copilot — Instructor-Led Training Material

